Lab 08

Parallel Computing Lab 08

SE4060-MPI-Part1

IT24103440



K.A.C.V. Kasthuriarachchi

IT24103440

Exercise 1 – Hello MPI

The program calls MPI_Init, obtains each process's rank with MPI_Comm_rank, obtains the processor name with MPI_Get_processor_name, prints a rank-dependent message, and closes MPI with MPI_Finalize. Rank 0 prints “Hello World”; other ranks print a normal-process message. The output order can vary because processes may execute concurrently.

Exercise 2 – Parallel addition

The program sums integers from 1 to 10,000,000. Each process calculates a local range, stores its partial sum in local_sum, and MPI_Reduce combines the partial sums using MPI_SUM at rank 0. The expected total is 50,000,005,000,000. Rank 0 reports process count, total sum, and elapsed time.

Exercise 3 – Monte Carlo estimation of π

Each process generates random points in the unit square and counts points satisfying $x^2 + y^2 \leq 1$, which fall inside a quarter circle. MPI_Reduce adds the local counts. Rank 0 estimates π as 4 × (points inside circle / total points). Because this is a random sampling method, the estimate varies between runs.

Exercise 4 – Performance graphs

The repository includes time_vs_processors.png and speedup.png. They compare the measured execution times and speedups for the parallel addition and Monte Carlo programs at 1, 2, 4, and 8 processes. The recorded results show lower execution time and higher speedup as process count increases.

Exercise 5 – Point-to-point communication

messages1.cc uses MPI_Ssend to synchronously send one integer (30) from rank 1 to rank 3; rank 3 receives and prints it. messaage2.cc uses MPI_Bsend to send an array of ten integers (0, 10, …, 90) after attaching a sufficiently sized MPI buffer. Rank 3 receives and prints the array. Both programs need at least four MPI processes to have ranks 1 and 3.

Exercise 6 – MPI_ANY_SOURCE

Worker processes send their local Monte Carlo inside-circle count to rank 0. Rank 0 receives from MPI_ANY_SOURCE, so messages can be collected in whichever order they arrive; status.MPI_SOURCE identifies the sender. Rank 0 sums the counts and computes the estimate of π.

Exercise 7 – Buffered send

This version uses MPI_Bsend for each worker's local count. Each sending process calculates the buffer size with MPI_Pack_size, adds MPI_BSEND_OVERHEAD, attaches the buffer, sends, detaches the buffer, and frees the memory. The receiver still uses MPI_ANY_SOURCE to collect worker results.

1. Recorded results

Exercise 2 – Parallel addition (sum of integers 1 to 10,000,000)

Processes	Execution time (s)	Speedup
1	0.010951	1.000
2	0.006239	1.755
4	0.004120	2.658
8	0.002156	5.079

Expected result: 50,000,050,000,000.

Exercise 3 – Monte Carlo π (10,000,000 total points)

Processes	Execution time (s)	Calculated π	Speedup
1	0.135876	3.1427376000	1.000
2	0.075890	3.1417704000	1.790
4	0.035822	3.1410696000	3.793
8	0.019808	3.1427052000	6.859

The π estimates are close to 3.14159, and the execution time decreases as the number of processes rises. Results may vary because the algorithm uses random samples.

2. Screenshot evidence

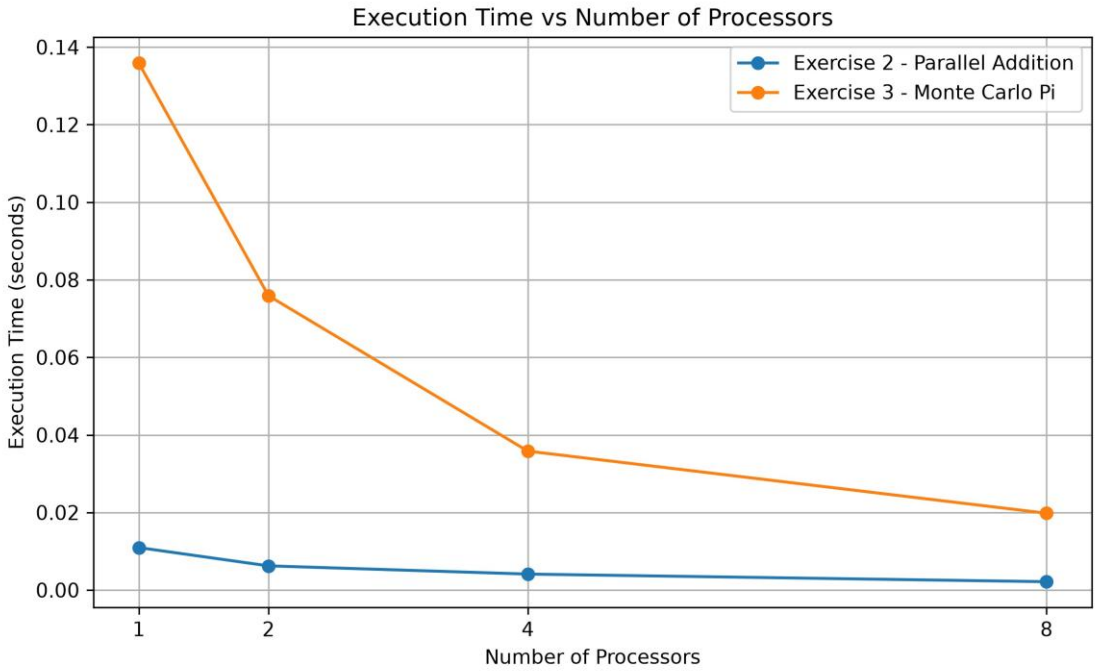


Figure 1. Execution time versus number of processors.

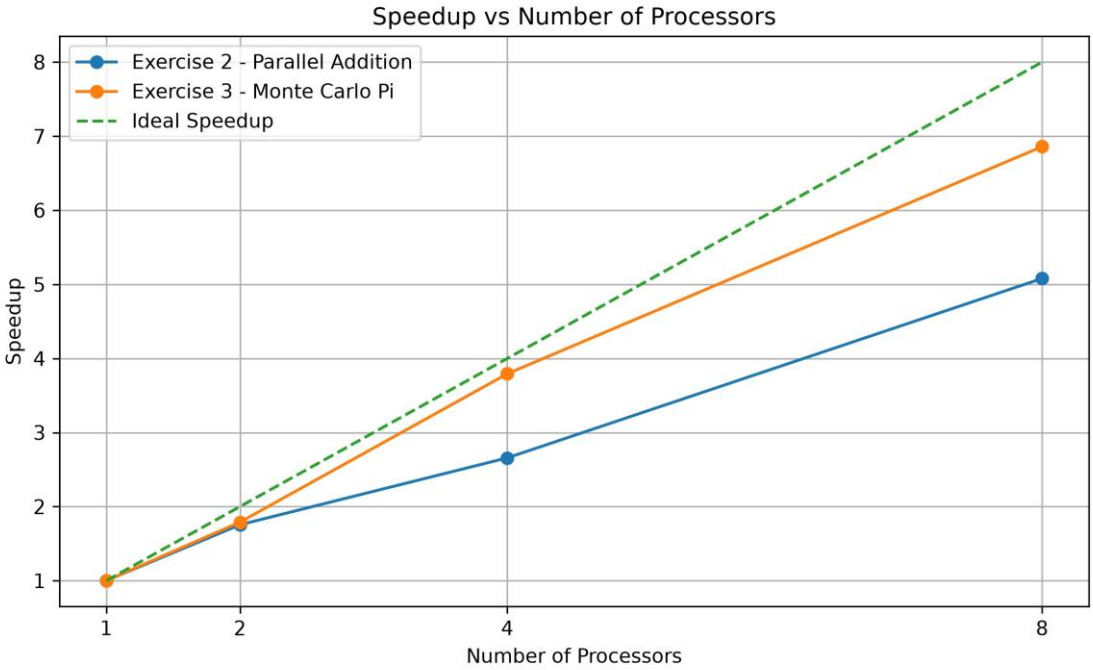


Figure 2. Speedup versus number of processors, including the ideal-speedup reference.

3. Key MPI concepts

Concept	Meaning
MPI_Init / MPI_Finalize	Start and end the MPI environment.
MPI_Comm_rank	Returns the calling process's rank within a communicator.
MPI_Comm_size	Returns the number of processes in a communicator.
MPI_Reduce	Combines values from all processes using an operation such as MPI_SUM and delivers root
MPI_Send / MPI_Recv	Standard point-to-point message send and receive.
MPI_Ssend	Synchronous send; completion requires a matching receive to have started.
MPI_Bsend	Buffered send; data is copied to an attached MPI buffer so the send can complete witho
MPI_ANY_SOURCE	Allows a receive to accept a message from any source rank; the actual source is availab MPI_Status.